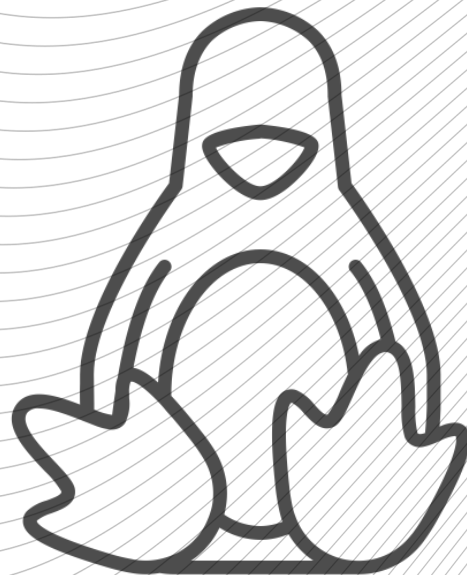


Exploring Linux

A Comprehensive Overview



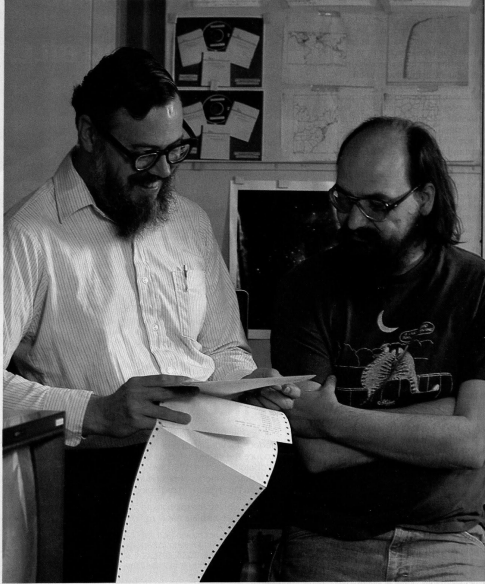
Agenda

- History of Unix & Linux
- Overview of Linux
 - Goals
 - Interfaces
 - The Shell
 - Utility Programs
 - Kernel Structure
- Processes in Linux
 - Process Management
 - Scheduling
 - Booting



HISTORY OF UNIX & LINUX

1982 AWARD FOR ACHIEVEMENT



DENNIS M. RITCHIE

KEN THOMPSON

- PDP-7 System challenges
- The Team for Multics
- The need of Time shared system, which allows Communal computing to all engineers
- UNIX
 - Unix was designed with a few key principles in mind, including simplicity, portability, and the idea of small, single-purpose utilities that can be combined to perform complex tasks.
 - It introduced the concept of a hierarchical file system and a shell for interacting with the system.

BIRTH OF LINUX



- Over time, Unix branched out into various versions and flavors, including BSD (Berkeley Software Distribution), System V, SunOS, AIX, HP-UX, and others. Each variant had its own features and implementations.
- In 1991, Linus Torvalds picks a Hobby project in Christmas Holidays
- Project was focussed to be created as Unix-like OS to be used for Personal computing.



GOALS OF LINUX

- Open Source
- Modularity
- Customizability
- Stability & Reliability
- Security
- Portability
- Collaboration
- Performance & Efficiency
- Availability
- Distribution

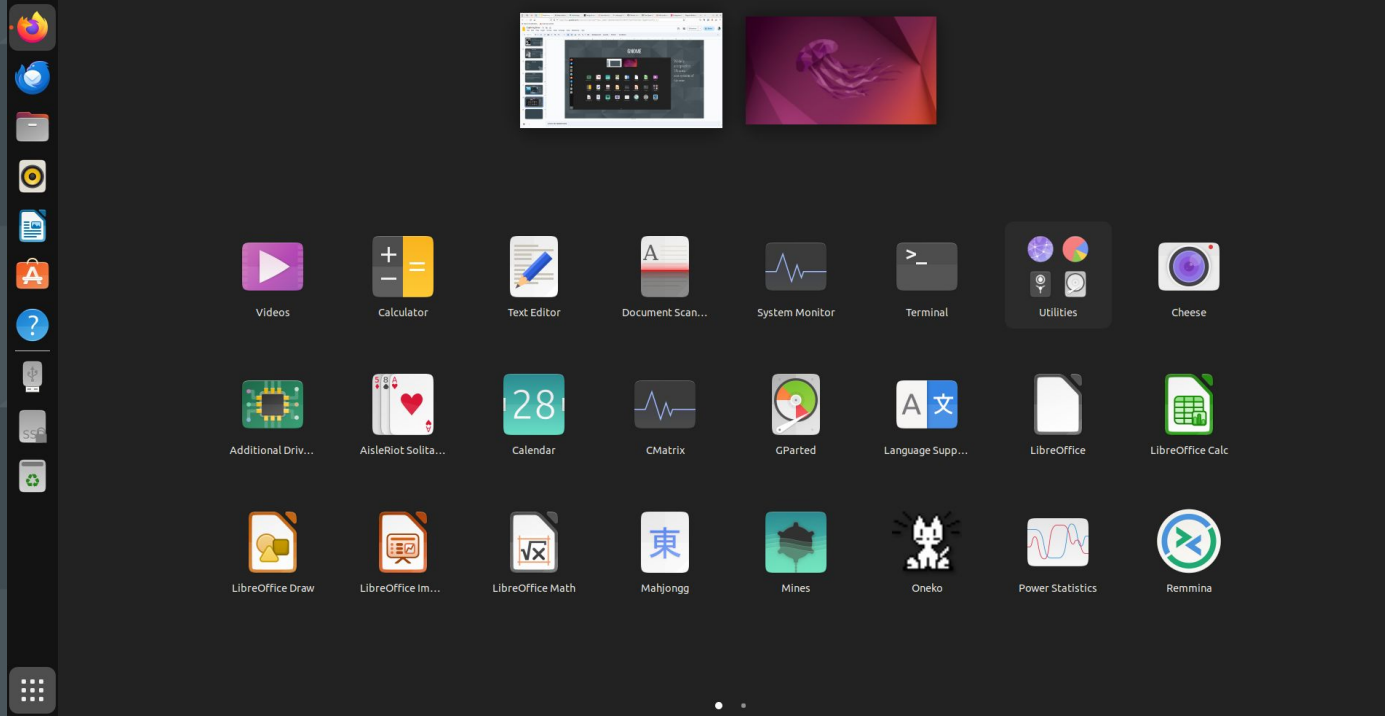


INTERFACES

Interfacing of Linux can be done with varied means depending upon the usecase:

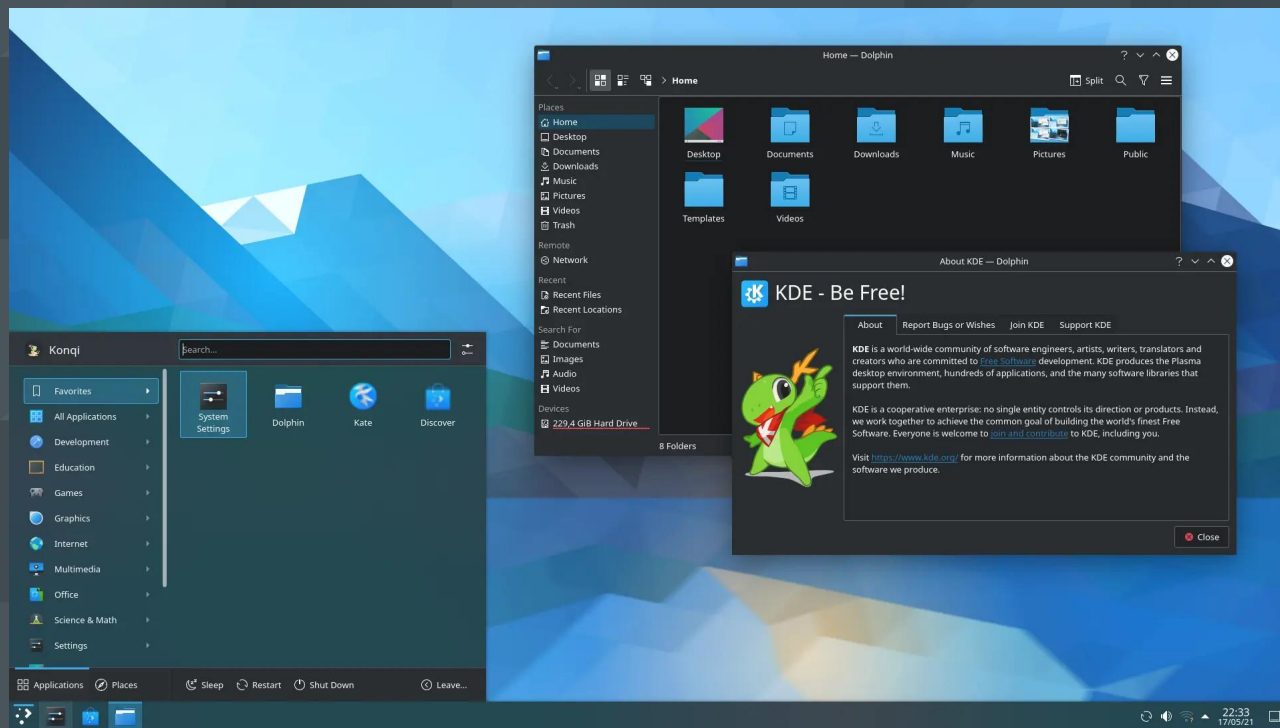
1. GUI - Unity, GNOME, Xmonad, Xfce
2. Command Line Interface (CLI) - BASH, Korn Shell, C Shell and Z Shell
3. Web UI - Mainly hosted for Monitoring and Maintenance purposes
4. Remote Access - Secure Shell (SSH), Secure File Transfer Protocol (SFTP), Remote Desktop Protocol (RDP/VNC)

GNOME



Widely
accepted in
Ubuntu
eco-system of
Gnome.

KDE Plasma



K Desktop Environment

- Friendly UI
- Was meant for TV set, later rolled out for Desktop for Great AV experience

XMONAD

```

> xmonad --recompile
XMonad will use build script at "/home/slot/.config/xmonad/build"
to recompile.
XMonad recompilation process exited with success!

-
>
[]

where
  threeCol = magnificercz' 1.3 $ ThreeColMid nmaster delta ratio
  tiled    = Tall nmaster delta ratio
  nmaster  = 1      -- Default number of windows in the master pane
  ratio    = 1/2    -- Default proportion of screen occupied by master pane
  delta    = 3/100  -- Percent of screen to increment by when resizing panes

myXmonbarPP :: PP
myXmonbarPP = def
  [ ppSep      = magenta " . "
  , ppTitle    = wrap (white "[") (white "]") . magenta . ppWindow
  , ppTitleUnfocused = wrap (lowWhite "[") (lowWhite "]") . blue . ppWindow
  , ppTitleSanitize = xmobarStrip
  , ppCurrent   = wrap " " "" . xmobarBorder "Top" "be9fd" 2
  , ppHidden    = white . wrap " " ""
  , ppHiddenNoWindows = lowWhite . wrap " " ""
  , ppUrgent    = red . wrap (yellow "!") (yellow "!")
  ]

where
  -- | Windows should have *some* title, which should not not exceed a
  -- sane length.
  ppWindow :: String -> String
  ppWindow = xmobarRaw . (\w -> if null w then "untitled" else w) . shorten 30
  blue, lowWhite, magenta, red, white, yellow :: String -> String
  magenta = xmobarColor "#ff79c6" ""
  blue    = xmobarColor "#bd93f9" ""
  white   = xmobarColor "#f8f8f2" ""
  yellow  = xmobarColor "#ffa83c" ""
  red     = xmobarColor "#ff5555" ""
  lowWhite = xmobarColor "#bbbbbb" ""

```

The screenshot shows a web browser window with the URL <https://hackage.haskell.org/package/pp>. The page title is "XMonad.Hooks.DynamicLog -". The main heading is "The PP type actually provides the user to customize the formatting of status information.", followed by the section "Constructors". Under "Constructors", there is a list of functions under the heading "PP":

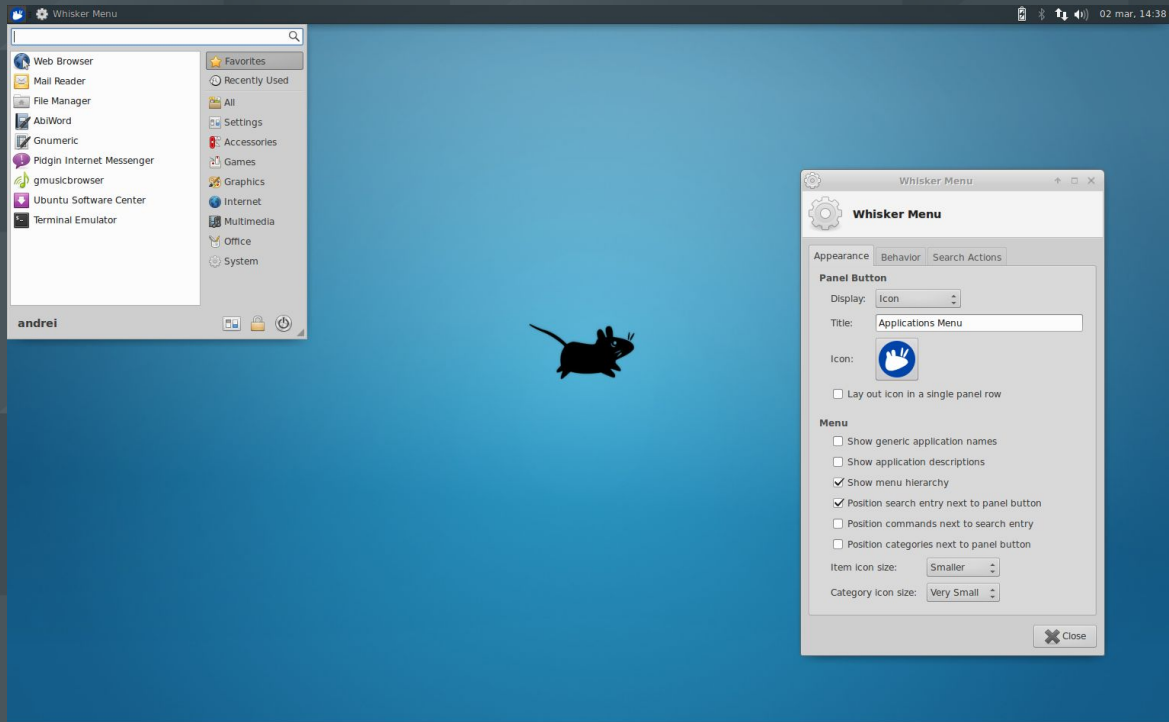
- `ppCurrent :: WorkspaceId -> String`
- `ppVisible :: WorkspaceId -> String`
- `ppHidden :: WorkspaceId -> String`
- `ppHiddenNoWindows :: WorkspaceId -> String`
- `ppVisibleNoWindows :: Maybe (WorkspaceId -> String)`
- `ppUrgent :: WorkspaceId -> String`
- `ppSep :: String`
- `ppTitleSep :: String`
- `ppTitle :: String -> String`
- `ppTitleSanitize :: String -> String`
- `ppLayout :: String -> String`
- `ppOrder :: [String] -> [String]`

To the right of each function signature, there are brief descriptions:

- `ppCurrent`: how to p currently
- `ppVisible`: how to p not focu (xineram
- `ppHidden`: how to p workspa windows
- `ppHiddenNoWindows`: how to p hidden v
- `ppVisibleNoWindows`: how to p visible w format t
- `ppUrgent`: urgent w
- `ppSep`: separatc different name, la
- `ppTitleSep`: separatc workspa
- `ppTitle`: window l
- `ppTitleSanitize`: escape / ppTitle
- `ppLayout`: layout n
- `ppOrder`: how to o sections function three for represen the layo window l have spe in ppExt also be i get them you can reverse displa y it could us

Xmonad a tiling window interface. Each application is adjusted in screen based on a ratio. Spiral, Grid, Columnar are some of the presets user can set.

XFCE



XFCE - Xfce aims to be fast and lightweight while still being visually appealing and easy to use.

Comically, The Slackware Linux distribution has nicknamed Xfce the "Cholesterol Free Desktop Environment", a loose interpretation of the initialism.

THE LINUX SHELL

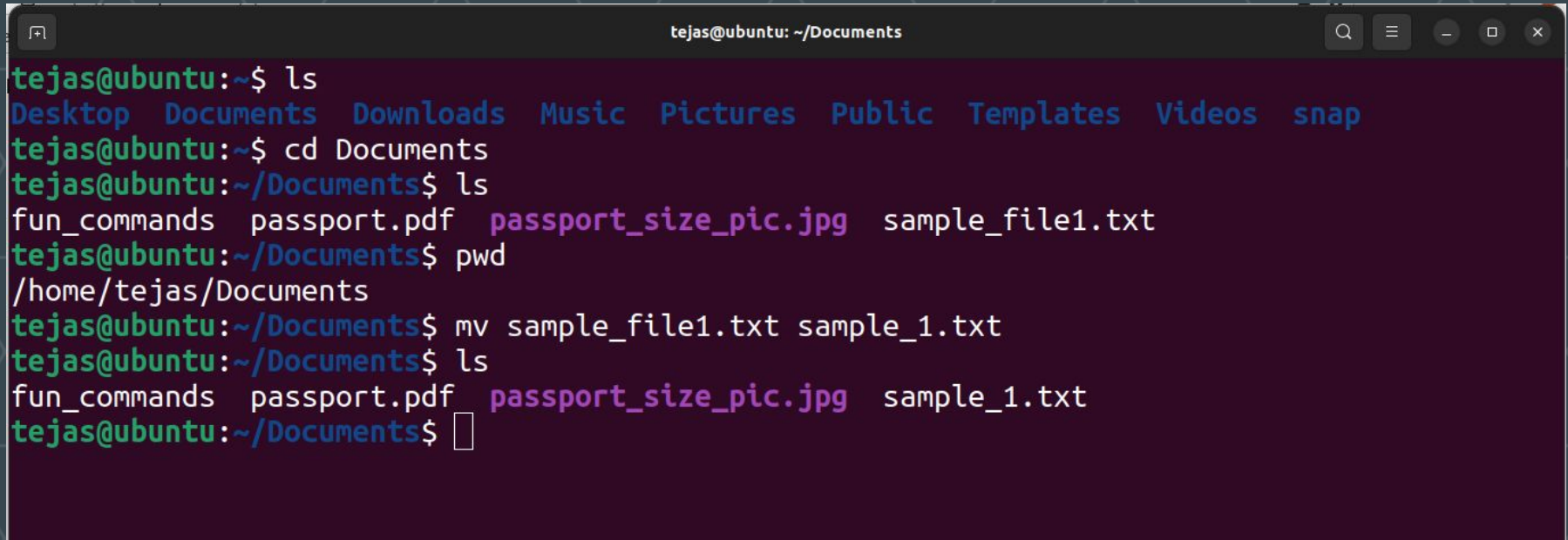
The Bash shell, short for the Bourne Again Shell, is one of the most widely used command-line interfaces in the Linux environment. It's the default shell for many Linux distributions and is also available on other Unix-like systems.

FEATURES:

- Interactive Shell
- Scripting Language
- Command History
- Tab Completion
- Job Control

1. File Management and Navigation

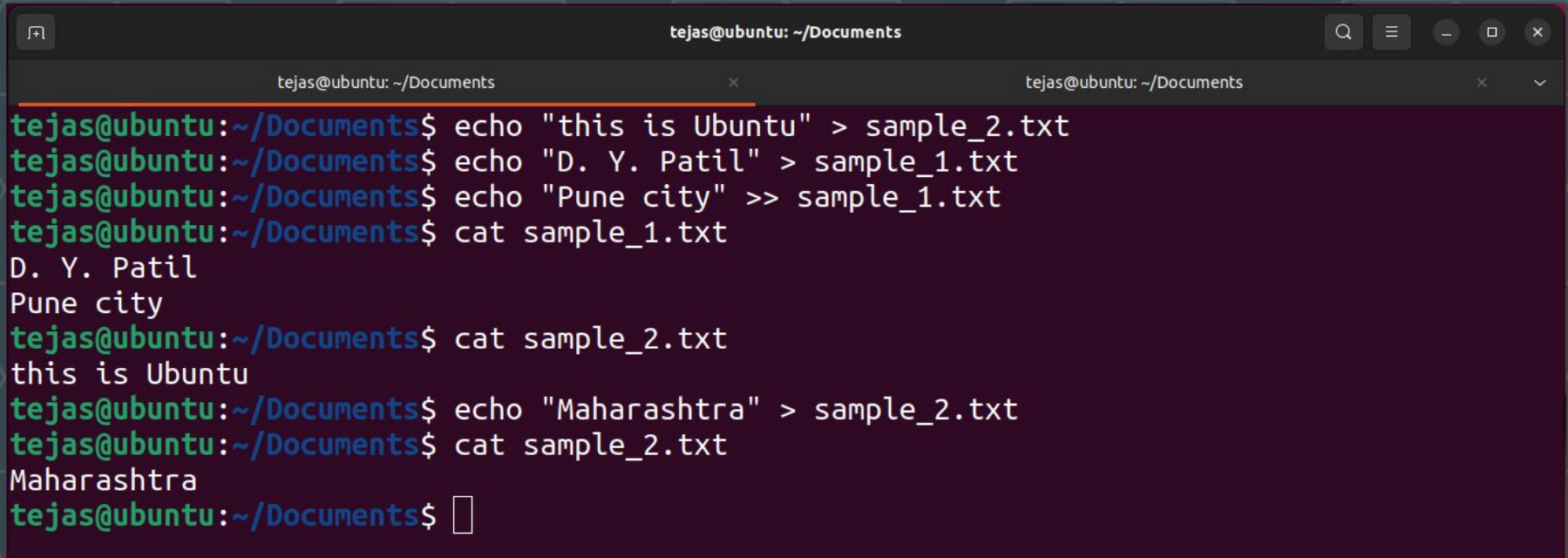
Basic commands like `cd <dir>` and `cd ..` are used to change directory one level up and down respectively. Following screencap shows a few more. More commands like: `cp`, `mv`, `rm`, `touch`, `mkdir` are also belong in this category.

A terminal window titled 'tejas@ubuntu: ~/Documents' with standard window controls. The terminal shows a sequence of commands and their outputs. The user starts in the home directory, lists files, moves into the 'Documents' directory, lists files again, changes the current directory to the full path of 'Documents', moves a file, and lists files once more.

```
tejas@ubuntu:~$ ls
Desktop  Documents  Downloads  Music  Pictures  Public  Templates  Videos  snap
tejas@ubuntu:~$ cd Documents
tejas@ubuntu:~/Documents$ ls
fun_commands  passport.pdf  passport_size_pic.jpg  sample_file1.txt
tejas@ubuntu:~/Documents$ pwd
/home/tejas/Documents
tejas@ubuntu:~/Documents$ mv sample_file1.txt sample_1.txt
tejas@ubuntu:~/Documents$ ls
fun_commands  passport.pdf  passport_size_pic.jpg  sample_1.txt
tejas@ubuntu:~/Documents$
```

2. Input and Output Redirection

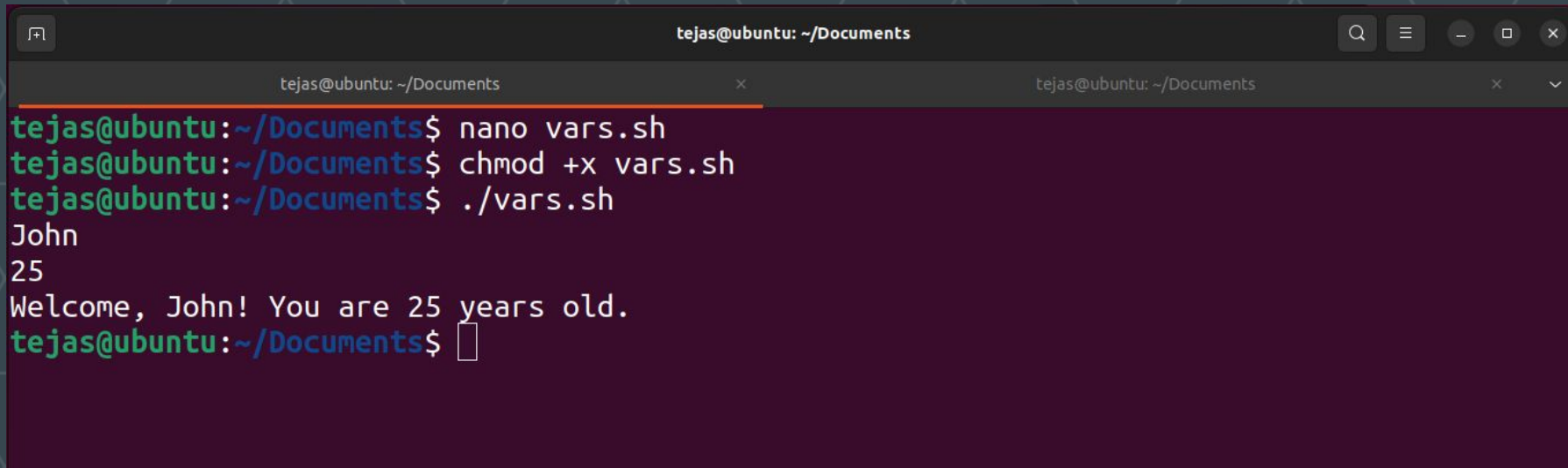
Allows directing command input/output to/from files, enabling users to save output to a file or use a file as input for a command.

A terminal window titled 'tejas@ubuntu: ~/Documents' with standard window controls. It shows a series of commands and their outputs. The first command 'echo "this is Ubuntu" > sample_2.txt' creates a file. The next two 'echo' commands append text to 'sample_1.txt'. The 'cat' command is used to display the contents of both files. The terminal text is as follows:

```
tejas@ubuntu:~/Documents$ echo "this is Ubuntu" > sample_2.txt
tejas@ubuntu:~/Documents$ echo "D. Y. Patil" > sample_1.txt
tejas@ubuntu:~/Documents$ echo "Pune city" >> sample_1.txt
tejas@ubuntu:~/Documents$ cat sample_1.txt
D. Y. Patil
Pune city
tejas@ubuntu:~/Documents$ cat sample_2.txt
this is Ubuntu
tejas@ubuntu:~/Documents$ echo "Maharashtra" > sample_2.txt
tejas@ubuntu:~/Documents$ cat sample_2.txt
Maharashtra
tejas@ubuntu:~/Documents$
```

3. Shell Variables

Bash uses variables to store information, modify behavior, and define the environment for the shell session. It supports conditional statements (if, else), loops (for, while), and functions, allowing for more complex scripting and logic.

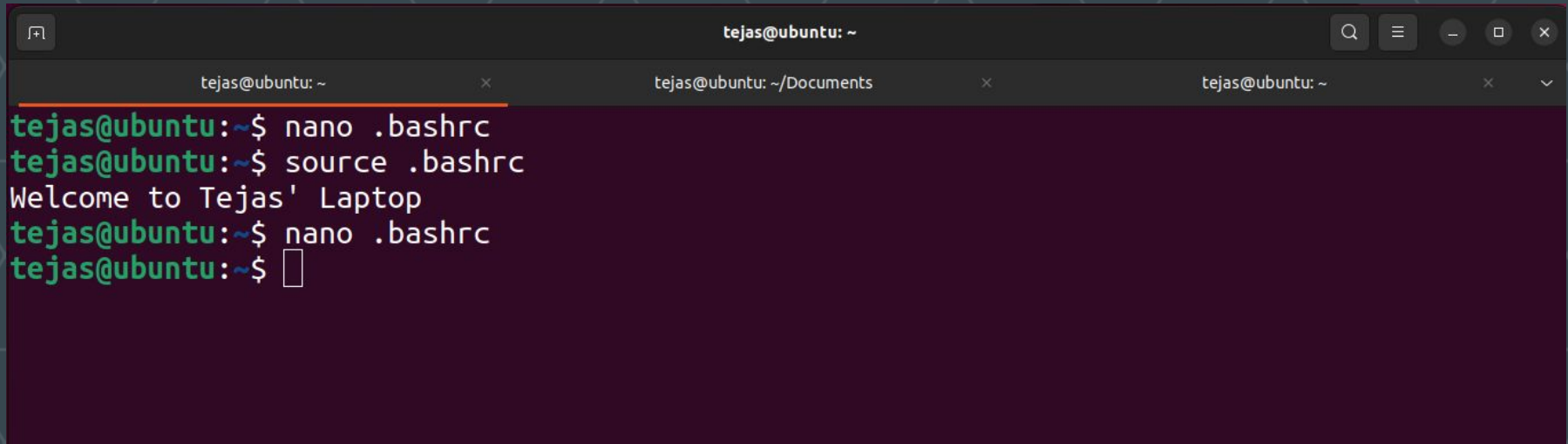
A terminal window titled 'tejas@ubuntu: ~/Documents' with standard window controls. It shows a sequence of commands and their output: creating a file 'vars.sh', making it executable, and running it. The script prints 'John' and '25', and then a formatted welcome message. The prompt is currently waiting for input.

```
tejas@ubuntu:~/Documents$ nano vars.sh
tejas@ubuntu:~/Documents$ chmod +x vars.sh
tejas@ubuntu:~/Documents$ ./vars.sh
John
25
Welcome, John! You are 25 years old.
tejas@ubuntu:~/Documents$
```

4. Configuration

These types of Kernel files are often used to customize a user behaviour in order to enact an OS routine/code. This feature puts the user in more control of their system.

Config files like `.bashrc`, `.bash_profile`, `.localrc`, `/etc/profile` and many more

A terminal window titled 'tejas@ubuntu: ~' with three tabs. The first tab is active and shows the following commands and output: 'nano .bashrc', 'source .bashrc', 'Welcome to Tejas' Laptop', 'nano .bashrc', and a blank prompt. The window has a dark theme and standard window controls.

```
tejas@ubuntu: ~$ nano .bashrc
tejas@ubuntu: ~$ source .bashrc
Welcome to Tejas' Laptop
tejas@ubuntu: ~$ nano .bashrc
tejas@ubuntu: ~$
```


5. Further Intermediate to Power Commands

Using CLI in Linux give you a lot more power, since the command set very extensive and has complex operations being done in simplest of commands with its attributes.

1. Listing with verbose details
2. Changing File Modes and RWX Permissions
3. Word Count & Mapping
4. Stream editing
5. Networking - SSH, FTP and more
6. Writing and Deploying custom kernel modules
7. ...and the list goes on

Let's try a few

6. Utility Programs

TEXT PROCESSING

```
sed 's/<exact word>/<word to replace>/'  
file.txt
```

GREP

```
grep '<case sens. word>' file_1.txt  
file_n.txt
```

COMPRESSING AND ARCHIVAL

```
tar -cvzf comp.tar.gz directory_1 #  
compress  
  
tar -xvzf comp.tar.gz # extract
```

SYS MONITORING

top, htop

NETWORKING

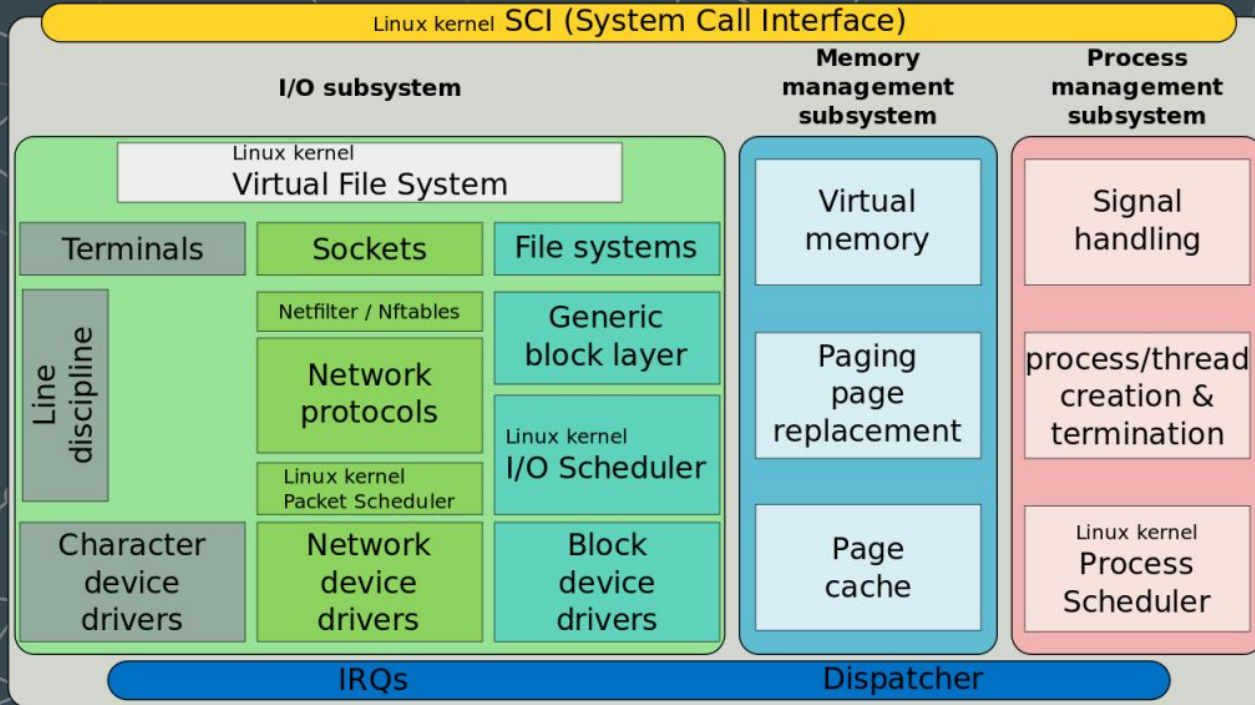
ps, netstat, ssh, scp, sftp

DISK OPS

du, df, fsdisk

Ls -blk, FUN CMDS, Startup Disk creator,
APT, Permissions

KERNEL STRUCTURE



The Linux kernel is a major component of the Linux operating system. It interfaces between your computer's hardware and its processes. It manages resources efficiently and communicates between the hardware and its processes. It is the core component of your Linux operating system.

KERNEL STRUCTURE

1. Process Management:

Scheduler: Manages CPU resources, schedules processes for execution, and handles multitasking.

Process Control Block (PCB): Data structure containing information about processes (e.g., PID, CPU state, memory allocation).

System Calls: Interface that allows user-space programs to request kernel services (e.g., file operations, memory allocation) securely.

2. Memory Management:

Virtual Memory: Manages memory addresses, implements paging, and provides memory protection.

Page Allocator: Allocates and deallocates memory pages for processes and the kernel.

Memory Mapping: Maps physical memory to virtual memory and supports various memory models (e.g., demand paging).

KERNEL STRUCTURE

3. File System:

File System Drivers: Provides support for various file systems (e.g., ext4, NTFS) and manages file I/O operations.

VFS (Virtual File System): Abstracts different file systems into a unified interface for applications.

4. Device Drivers:

Device Abstraction Layer: Manages hardware devices and provides a unified interface for accessing them.

Kernel Space Drivers: Implement device-specific operations and manage interactions with hardware.

5. Networking:

Network Stack: Implements protocols (TCP/IP, UDP), socket management, and network device drivers.

Packet Handling: Handles packet routing, filtering, and network I/O.

6. Kernel Services:

Interrupt Handlers: Manage hardware interrupts and provide timely responses to hardware events.

Timer Handling: Manages system time, timers, and scheduling of delayed operations.

Synchronization and Locking: Provides mechanisms like semaphores, mutexes, and spinlocks for synchronization and concurrency control.

KERNEL STRUCTURE

7. Kernel API and Subsystems:

- Kernel API: Defines interfaces used by user-space applications to interact with the kernel.

- Subsystem Modules: Additional kernel components providing specialized functionalities (e.g., security, power management, virtualization).

8. Modular Design:

- Loadable Kernel Modules (LKMs): Allows dynamic loading and unloading of kernel modules at runtime, extending kernel functionality without recompiling the entire kernel.

Process Management

Types of Processes in Linux

In Linux processes can be of two types:

- **Foreground Processes:** depend on the user for input also referred to as interactive processes
- **Background Processes:** run independently of the user referred to as non-interactive or automatic processes



Process Management

Process States in Linux

A process in Linux can go through different states after it's created and before it's terminated. These states are:

- Running
- Sleeping
 - Interruptible sleep
 - Uninterruptible sleep
- Stopped
- Zombie

Process States explanation:

- A process in running state means that it is running or it's ready to run.
- The process is in a sleeping state when it is waiting for a resource to be available.
- A process in Interruptible sleep will wake up to handle signals, whereas a process in Uninterruptible sleep will not.
- A process enters a stopped state when it receives a stop signal.
- Zombie state is when a process is dead but the entry for the process is still present in the table.

Process Management System Calls

1. Process Creation:

Forking: The primary method to create a new process. The `fork()` system call is used to duplicate an existing process, resulting in two identical processes (parent and child) sharing the same code, data, and resources.

Execution: After forking, the `exec()` system call is often used to replace the child process's memory space with a new program's code and data.

2. Process States:

Running: The process is currently using the CPU.

Ready: The process is prepared to run but waiting for CPU time.

Blocked: The process is waiting for an event (e.g., I/O operation) to complete.

Process Management System Calls

3. Process Control Block (PCB):

- **Data Structure:** Each process is represented by a PCB containing essential information like process ID (PID), program counter, CPU registers, memory allocation details, etc.

4. Process Scheduling:

- **Scheduler:** The kernel's scheduler determines which process gets CPU time and for how long, ensuring fairness and efficient resource utilization.
- **Scheduling Algorithms:** Various algorithms (e.g., round-robin, priority-based scheduling) govern how processes are selected for execution.

5. Context Switching:

- **Switching between Processes:** When the scheduler selects a new process to run, a context switch occurs. The current process's state is saved, and the CPU state is loaded with the state of the selected process.

6. Inter-Process Communication (IPC):

- **Mechanisms:** Processes can communicate and synchronize using IPC mechanisms like shared memory, message passing, semaphores, pipes (Ex. `mkfifo my_pipe`), and sockets.

Process Management System Calls

7. Process Termination:

- **Exit:** Processes can terminate voluntarily by calling the `exit()` system call or forcefully using signals.
- **Cleanup:** Resources associated with the process, such as memory and open files, are released upon termination.

8. Process Hierarchy:

- **Parent-Child Relationship:** A parent process can create child processes, forming a hierarchy. Child processes inherit certain attributes from their parent.

9. Zombie and Orphan Processes:

- **Zombies:** Processes that have completed execution but still have an entry in the process table until the parent retrieves their exit status.
- **Orphans:** Child processes that continue to run even after the parent process has terminated. They are usually adopted by the `init` process.

Efficient process management is essential for system stability, resource allocation, and multitasking capabilities, allowing multiple programs to run concurrently and ensuring smooth execution of tasks within an operating system like Linux.

PROCESS SCHEDULING

Process scheduling in Linux refers to the method by which the Linux kernel manages and allocates CPU time to various processes running on the system. The scheduler in Linux determines which process gets to use the CPU and for how long, aiming to provide fairness, responsiveness, and efficient utilization of system resources.

PROCESS SCHEDULING

1. Scheduler Classes in Linux:

- Completely Fair Scheduler (CFS):
 - a. Default Scheduler: CFS is the default scheduler in modern Linux kernels
 - b. Fairness: It aims to provide fairness by equally distributing CPU time among active processes, ensuring each process gets a fair share of CPU time.
 - c. Dynamic Priority: Uses a virtual runtime model to maintain process priority and determines the next task to execute based on this runtime.
- b. Real-Time Scheduler:
 - 1. Specialized for Real-Time Tasks: Provides real-time scheduling capabilities for tasks requiring strict timing constraints.
 - 2. Prioritization: Allows tasks to have different priorities and guarantees timely execution for high-priority real-time tasks.

PROCESS SCHEDULING

2. Process Scheduling Policies:

a. Time-Sharing (TS) Policy:

1. CFS Implementation: The CFS scheduler primarily follows the time-sharing policy.
2. Equal CPU Allocation: Prioritizes fair distribution of CPU time among active processes for responsiveness and fairness.

b. Real-Time (RT) Policy:

1. Fixed Priorities: Assigns fixed priorities to real-time tasks, ensuring timely execution for critical tasks with specific deadlines.

PROCESS SCHEDULING

3. Scheduling Classes:

1. Scheduling Classes in CFS:

- a. SCHED_NORMAL: Used for standard processes, providing fair time-sharing scheduling.
- b. SCHED_BATCH: Optimized for background tasks, allowing them to run with lower priority to avoid impacting foreground tasks.
- c. SCHED_IDLE: Lowest priority class, suitable for extremely low-priority background tasks.

2. Scheduling Classes in Real-Time:

- a. SCHED_FIFO: Provides a First-In, First-Out scheduling policy for real-time tasks, where tasks with the highest priority execute first.
- b. SCHED_RR (Round Robin): Similar to FIFO but with time-sliced scheduling for real-time tasks.

PROCESS SCHEDULING

4. Process States and Context Switching:

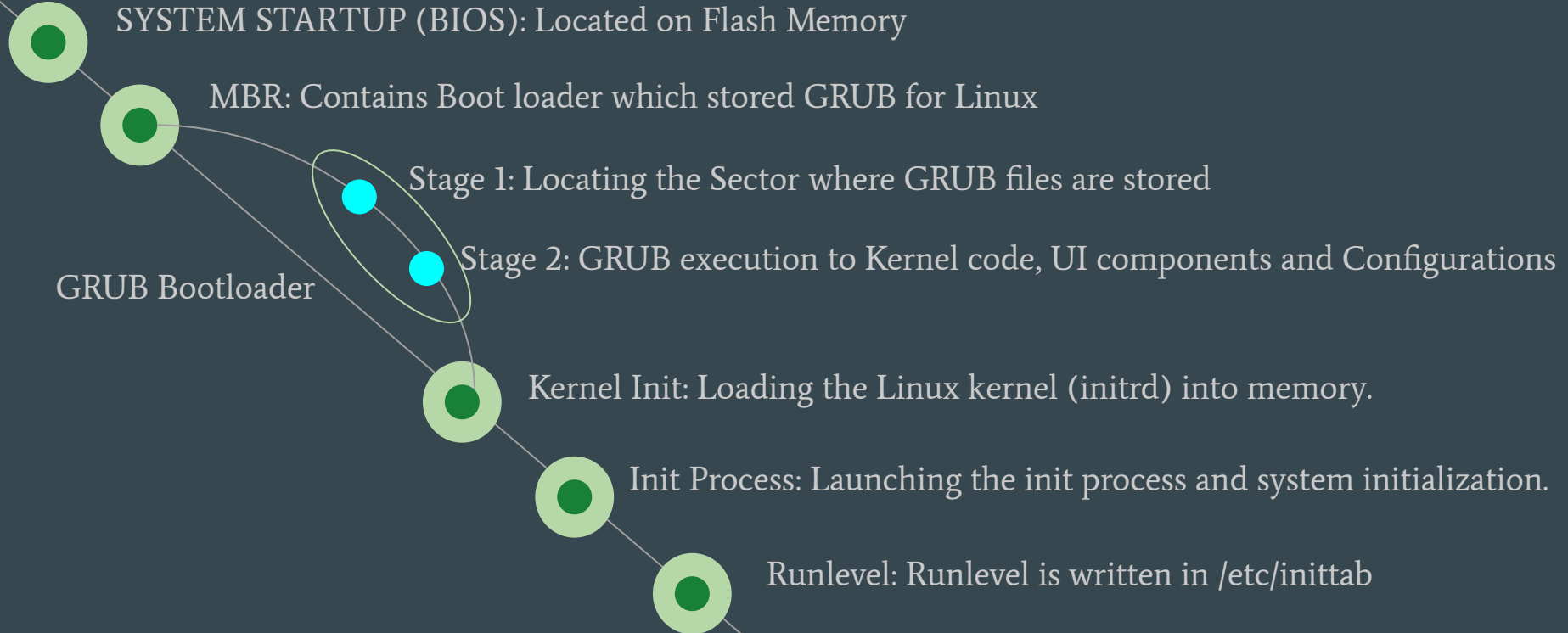
1. Process States:

- a. Running: Currently using CPU.
- b. Ready: Ready to run but waiting for CPU time.
- c. Blocked: Waiting for an event (e.g., I/O) to occur.

2. Context Switching:

- a. Switching between Processes: Occurs when the scheduler decides to give CPU time to another process. Saves the current process state and loads the state of the next process.
- b. Overhead: Context switching incurs overhead but is necessary for multitasking and maintaining fairness.

BOOTING



Runlevels

1. **N (Target not set)**: Indicates that a specific target is not set or that the system is not using SysVinit-style runlevels.
2. **0 (halt.target)**: System halt or shutdown state.
3. **1 (rescue.target/emergency.target)**: Single-user mode for system maintenance, equivalent to runlevel 1.
4. **2 (multi-user.target)**: Multi-user mode without networking.
5. **3 (multi-user.target)**: Multi-user mode with networking but without a graphical interface.
6. **4 (user-defined)**: Not predefined, user-defined target.
7. **5 (graphical.target)**: Multi-user mode with networking and graphical interface (GUI), equivalent to runlevel 5.
8. **6 (reboot.target)**: System reboot.

Questions & Feedback

Thank you

Tejas Rawal
tejasofficial@pm.me

References:

1. Linux Command Line and Shell Scripting Bible by Richard Blum
2. Digital Ocean
3. Practical Linux infrastructure by Syed Ali

Artifacts:

1. Icons from TheNounProject
2. Backdrops from PNGWing
3. Language summarization from ChatGPT